上海科技大学
ShanghaiTech University

# CS240 Algorithm Design and Analysis

## Lecture 12

## NP and Computational Intractability (Cont.)

Quan Li
Fall 2025
2025.10.28

# NP-Completeness

# NP-Complete

- **NP-complete.** A problem Y in NP with the property that for every problem X in NP, $X \leq_p Y$

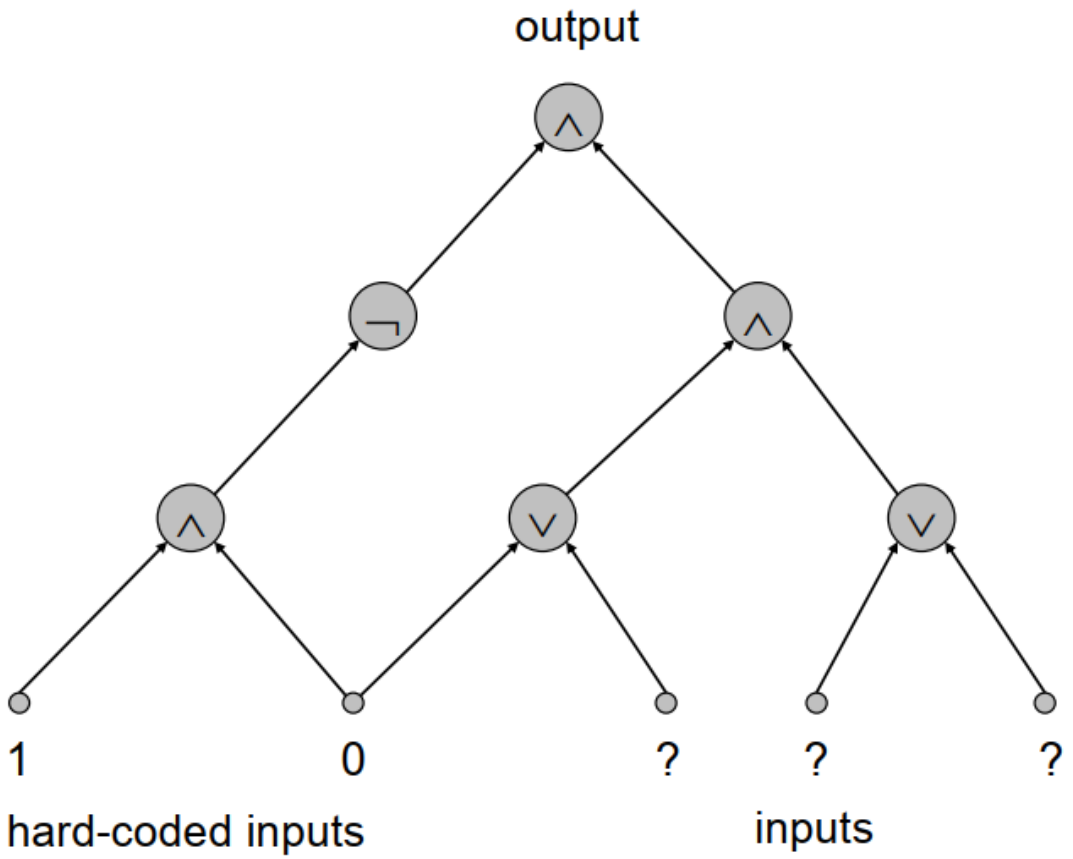
- **Theorem.** Suppose Y is an NP-complete problem. Then Y is solvable in poly-time iff P = NP
- **Pf.** ← If P = NP then Y can be solved in poly-time since Y is in NP
- **Pf.** → Suppose Y can be solved in poly-time
  - Let X be any problem in NP. Since $X \leq_p Y$ we can solve X in poly-time. This implies $NP \subseteq P$
  - We already know $P \subseteq NP$. Thus P = NP

- **Fundamental question.** Do there exist "natural" NP-complete problems?

# Circuit Satisfiability

- **CIRCUIT-SAT.** Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?

yes: 1 0 1



# The "First" NP-Complete Problem

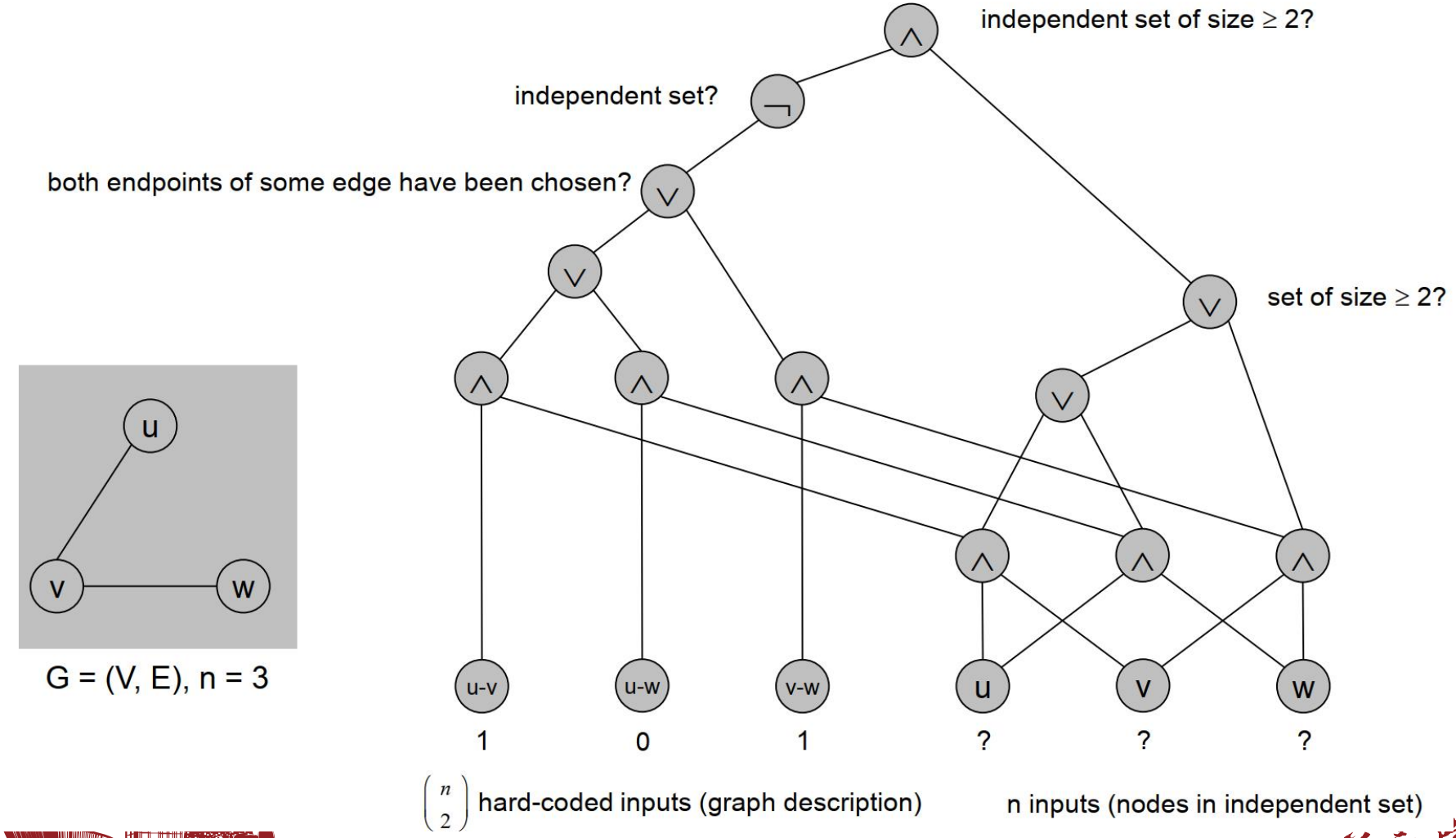
- **Theorem.** CIRCUIT-SAT is NP-complete. [Cook 1971, Levin 1973]
- **Pf.** (sketch)
  - Clearly, CIRCUIT-SAT is in NP
  - Any algorithm that takes a fixed number of bits n as input and produces a yes/no answer can be represented by such a circuit. Moreover, if algorithm takes poly-time, then circuit is of poly-size

Sketchy part of proof; fixing the number of bits is important, and reflects basic distinction between algorithms and circuits

  - Consider some problem X in NP. It has a poly-time certifier C(s, t). To determine whether s is in X, need to know if there exists a certificate t of length p(|s|) such that C(s, t) = yes
  - View C(s, t) as an algorithm on |s| + p(|s|) bits (input s, certificate t) and convert it into a poly-size circuit K
    - First |s| bits are hard-coded with s
    - Remaining p(|s|) bits represent (unknown) bits of t
  - Circuit K is satisfiable iff ∃t, C(s, t) = yes

# Example

- **Ex.** Construction below creates as circuit K whose inputs can be set so that K outputs true iff graph G has an independent set of size ≥ 2



# Establishing NP-Completeness

- **Remark.** Once we establish first "natural" NP-complete problem, others fall like dominoes
- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that $X \leq_p Y$

- **Theorem.** If X is NP-complete problem and Y is a problem in NP with the property that $X \leq_p Y$, then Y is NP-complete
- **Pf.** Let W be any problem in NP. Then $W \leq_p X \leq_p Y$
  - By transitivity, $W \leq_p Y$
  - Hence Y is NP-complete

By definition of NP-complete

By assumption

# 3-SAT is NP-Complete



**Review of 3-SAT**

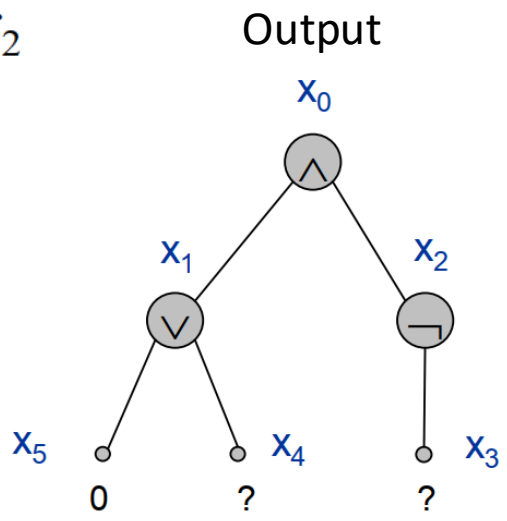
- **Literal:** A Boolean variable or its negation $x_i$ or $\overline{x_i}$
- **Clause:** A disjunction of literals $C_j = x_1 \vee \overline{x_2} \vee x_3$
- **Conjunctive normal form:** A propositional formula **Φ** that is the conjunction of clauses $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$
- **SAT.** Given CNF formula **Φ**, does it have a satisfying truth assignment?
- **3-SAT:** SAT where each clause contains exactly 3 literals

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

Yes: $x_1$ = true, $x_2$ = true, $x_3$ = false, $x_4$ = false.

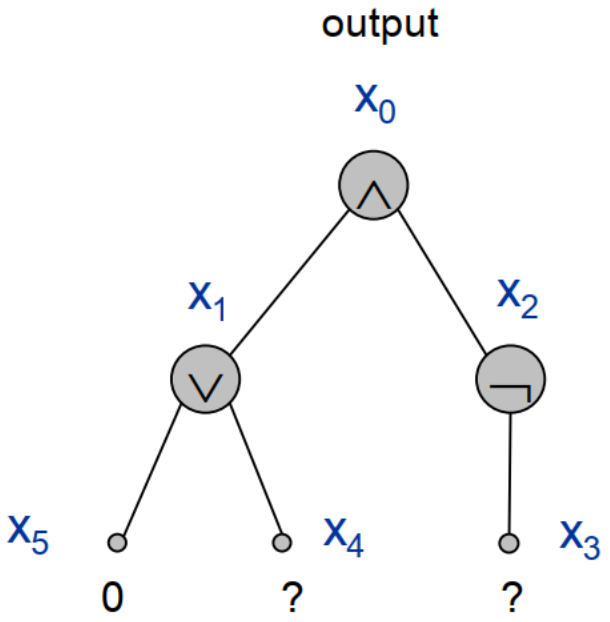
# 3-SAT is NP-Complete

- **Theorem.** 3-SAT is NP-complete
- **Pf.** Suffices to show that CIRCUIT-SAT $\leq_p$ 3-SAT since 3-SAT is in NP
  - Let K be any circuit. We can construct an instance of 3-SAT **Φ**
  - Step 1: Create a 3-SAT variable $x_i$ for each circuit element i
  - Step 2: Make circuit compute correct values at each node:
    - $X_2 = \neg X_3$ → add 2 clauses: $x_2 \vee x_3$ , $\overline{x_2} \vee \overline{x_3}$
    - $X_1 = X_4 \vee X_5$ → add 3 clauses: $x_1 \vee \overline{x_4}$, $x_1 \vee \overline{x_5}$ , $\overline{x_1} \vee x_4 \vee x_5$
    - $X_0 = X_1 \wedge X_2$ → add 3 clauses: $\overline{x_0} \vee x_1$, $\overline{x_0} \vee x_2$, $x_0 \vee \overline{x_1} \vee \overline{x_2}$
  - Step 3: Hard-coded input values and output value
    - $X_5 = 0$ → add 1 clauses: $\overline{x_5}$
    - $X_0 = 1$ → add 1 clauses: $x_0$
  - Final step: turn clauses of length < 3 into clauses of length exactly 3



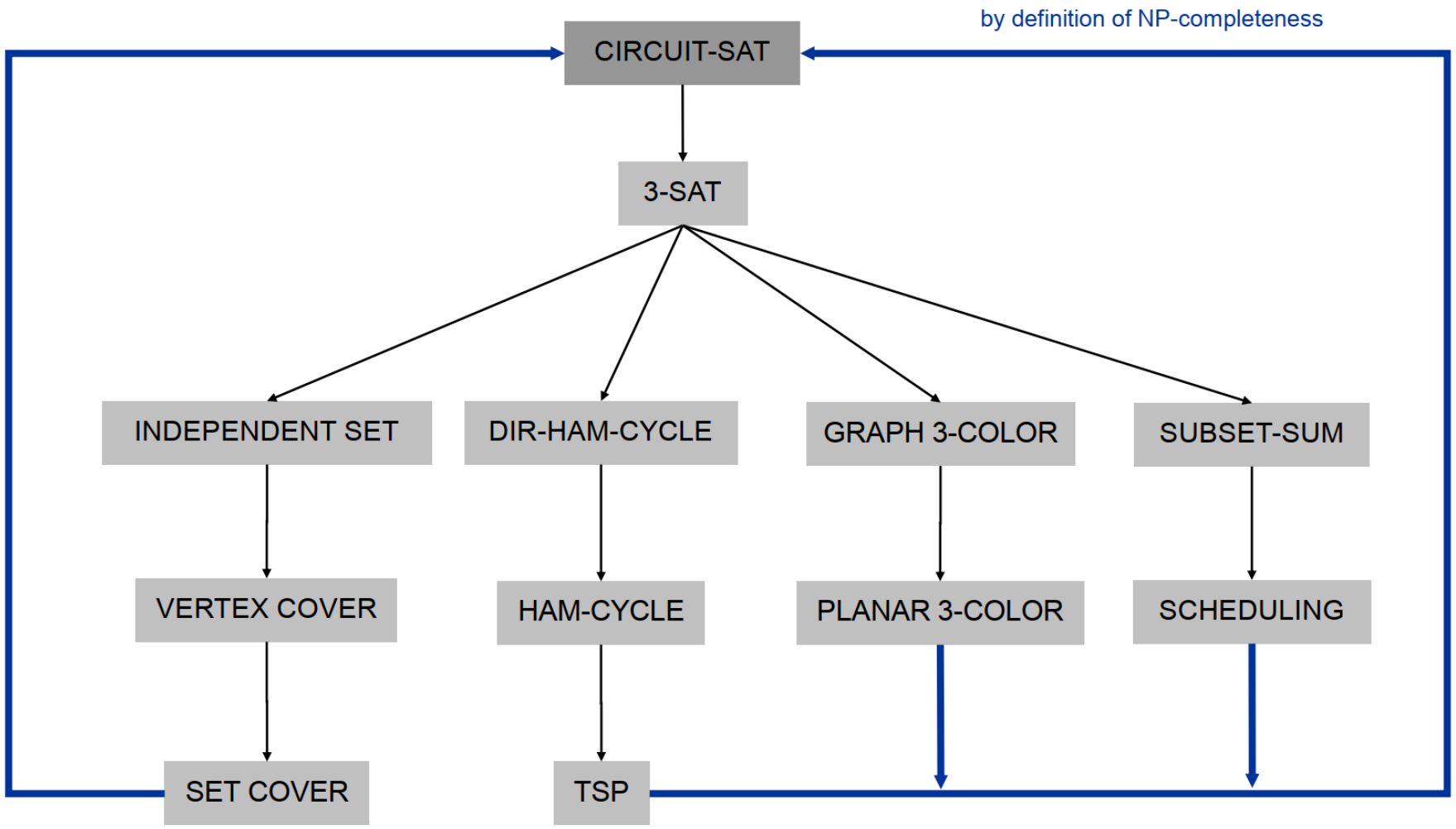
# 3-SAT is NP-Complete

- **Lemma.** $\phi$ is satisfiable iff the inputs of K can be set s.t. it outputs 1
- **Pf. ←**
  - Suppose there are inputs of K s.t. it outputs 1
  - Compute the values of all the nodes in K
  - According to our construction, this set of values satisfies $\phi$
- **→**
  - Suppose $\phi$ is satisfiable
  - The variable assignment of $\phi$ specifies the values of all the nodes in K
  - Our construction ensures that all the nodes in K are correctly computed and the output is 1



# NP-Completeness

- **Observation.** All problems below are NP-complete and polynomial reduce to one another!



# Some NP-Complete Problems

- **Six basic genres of NP-complete problems and paradigmatic examples**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - Partitioning problems: 3D-MATCHING, 3-COLOR
  - Numerical problems: SUBSET-SUM, KNAPSACK

- **Practice.** Most NP problems are known to be either in P or NP-complete

- **Notable exceptions**. Factoring, graph isomorphism, Nash equilibrium
- **Theorem.** [Ladner 1975] Unless P = NP, there exist problems in NP that are neither in P nor NP-complete

# Extent and Impact of NP-Completeness



- **Extent of NP-completeness.** [Papadimitriou 1995]
  - 6,000 citations per year (title, abstract, keywords)
  - More than “compiler”, “operating system”, “database”
  - Prime intellectual export of CS to other disciplines
  - Broad applicability and classification power
  - “Captures vast domains of computational, scientific, mathematical endeavors, and seems to roughly delimit what mathematicians and scientists had been aspiring to compute feasibly”

- **NP-completeness can guide scientific inquiry**
  - 1926: Ising introduces simple model for phase transitions
  - 1944: Onsager finds closed-form solution to 2D-Ising
  - 19xx: Feynman and other top minds seek 3D solution
  - 2000: Istrial proves 3D-Ising is NP-complete; thus search for closed-form solution to 3D-Ising appears doomed

# More Hard Computational Problems

- **Aerospace engineering:** optimal mesh partitioning for finite elements.
- **Biology:** protein folding.
- **Chemical engineering:** heat exchanger network synthesis.
- **Civil engineering:** equilibrium of urban traffic flow.
- **Economics:** computation of arbitrage in financial markets with friction.
- **Electrical engineering:** VLSI layout.
- **Environmental engineering:** optimal placement of contaminant sensors.
- **Financial engineering:** find minimum risk portfolio of given return.
- **Game theory:** find Nash equilibrium that maximizes social welfare.
- **Genomics:** phylogeny reconstruction.
- **Mechanical engineering:** structure of turbulence in sheared flows.
- **Medicine:** reconstructing 3-D shape from biplane angiocardiogram.
- **Operations research:** optimal resource allocation.
- **Physics:** partition function of 3-D Ising model in statistical mechanics.
- **Politics:** Shapley-Shubik voting power.
- **Pop culture:** Minesweeper consistency.
- **Statistics:** optimal experimental design.

# Terminology

- **NP-complete.** A problem in NP such that every problem in NP polynomial reduces to it
- **NP-hard.** [Bell Labs, Steve Cook, Ron Rivest, Sartaj Sahni]

  A problem such that every problem in NP reduces to it

Stephen Cook

## Stephen Cook

- 1971 paper "The Complexity of Theorem Proving Procedures"
  - formalized the notions of polynomial-time reduction and NP-completeness
  - proved the existence of an NP-complete problem (SAT)
- Bio
  - Joined UC Berkeley in 1966 but was denied tenure in 1970
  - Joined U of Toronto in 1970
  - 1982 Turing award

Richard Karp

"It is to our everlasting shame that we were unable to persuade the math department to give him tenure." -- Richard Karp (1985 Turing award)

# Co-NP and the Asymmetry of NP

# Asymmetry of NP

- **NP.** Decision problems for which there is a poly-time certifier
- **Ex.** SAT, HAM-CYCLE, COMPOSITES
- **Asymmetry of NP.** We only need to have short proofs of yes instances
- It may not be easy to show proofs of no instances
- **Ex 1.** SAT vs. TAUTOLOGY
  - Can prove a CNF formula is satisfiable by giving such an assignment
  - How could we prove that a formula is **not** satisfiable?
- **Ex 2.** HAM-CYCLE vs. NO-HAM-CYCLE
  - Can prove a graph is Hamiltonian by giving such a Hamiltonian cycle
  - How could we prove that a graph is not Hamiltonian?
- **Q.** How do we classify TAUTOLOGY?
  - SAT is NP-complete and SAT $\equiv_p$ TAUTOLOGY
  - But TAUTOLOGY is not even known to be in NP

# NP and co-NP

- **NP.** Decision problem for which there is a poly-time certifier
- **Ex.** SAT, HAM-CYCLE, COMPOSITES

- **Def.** Given a decision problem X, its complement is $\overline{X}$: same problem with the yes and no answers reverse

- **Ex.** $\overline{X}$ {0, 1, 4, 6, 8, 9, 10, 12, 14, 15, …}

  X = {2, 3, 5, 7, 11, 13, 17, 23, 29, …}

- **Co-NP.** Complements of decision problems in NP
  - For no instance, there is a succinct disqualifier
- **Ex.** TAUTOLOGY, NO-HAM-CYCLE, PRIMES

# NP = co-NP?

- **Fundamental question.** Does NP = co-NP?
  - Do yes instances have succinct certificates iff no instances do?
  - Consensus opinion: no

- **Theorem.** If NP ≠ co-NP, then P ≠ NP
- **Pf idea**
  - P is closed under complementation
  - If P = NP, then NP is closed under complementation
  - In other words, NP = co-NP
  - This is the contrapositive of the theorem

# Good Characterizations

- **Good characterization.** [Edmonds 1965] NP ⋂ co-NP
  - If problem X is in both NP and co-NP, then:
    - For yes instance, there is a succinct certificate
    - For no instance, there is a succinct disqualifier
  - Provides conceptual leverage for reasoning about a problem

- **Ex.** Given a bipartite graph, is there a perfect matching
  - If yes, can exhibit a perfect matching
  - If no, can exhibit a set of nodes S such that $|N(S)| < |S|$

# Good Characterizations

- **Observation.** $P \subseteq NP \cap co\text{-}NP$
- **Fundamental open question.** Does $P = NP \cap co\text{-}NP$?
  - Mixed opinions
  - Many examples where problem found to have a non-trivial good characterization, but only years later discovered to be in P
    - Linear programming [Khachiyan, 1979]
    - Primality testing [Agrawal-Kayal-Saxena, 2002]

- **Fact.** Factoring is in $NP \cap co\text{-}NP$, but not known to be in P

If poly-time algorithm for factoring, can break RSA cryptosystem

# FACTOR is in NP ⋂ co-NP

- **FACTORIZE.** Given an integer x, find its prime factorization
- **FACTOR.** Given two integers x and y, does x have a nontrivial factor less than y?
- **Theorem.** FACTOR $\equiv_p$ FACTORIZE.
- **Pf.** Binary search to find a factor, divide the factor and repeat

- **Theorem.** FACTOR is in NP ∩ co-NP
- **Pf.**
  - Certificate: a factor p of x that is less than y
  - Disqualifier: the prime factorization of x (where each prime factor is ≥ y), along with a Pratt's certificate that each factor is prime

# Primality Testing and Factoring



- **Q: Is PRIMES in P?**
  - Yes! (Agrawal-Kayal-Saxena, 2002)

- **Q: Is FACTOR in P?**
  - Consensus: No

- **RSA cryptosystem**
  - Based on dichotomy between complexity of two problems
  - To use RSA, must generate large primes efficiently
  - To break RSA, must find efficient factoring algorithm

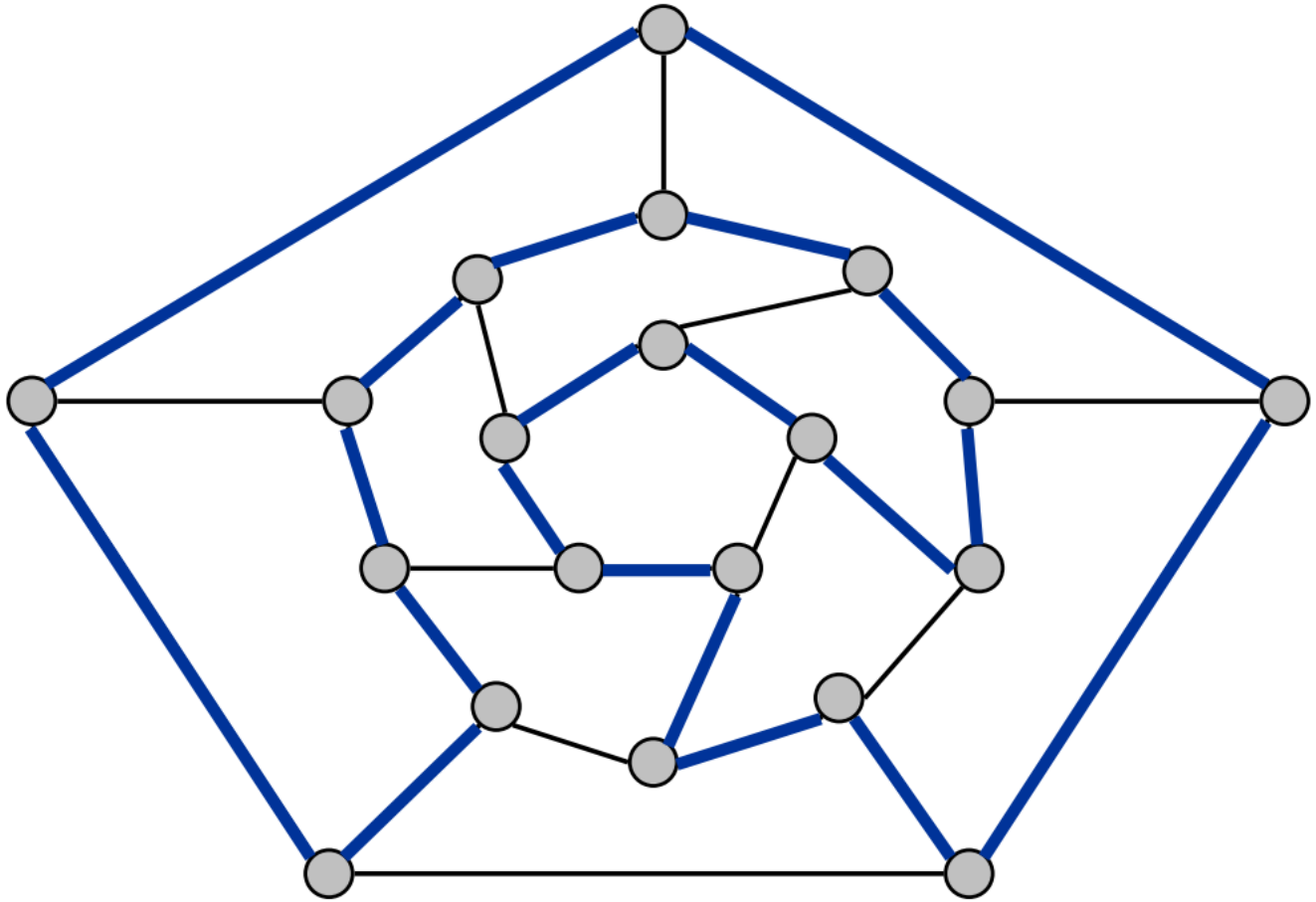
# Sequencing Problems



- **Basic genres**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - **Sequencing problems: HAMILTONIAN-CYCLE, TSP**
  - Partitioning problems: 3D-MATCHING, 3-COLOR
  - Numerical problems: SUBSET-SUM, KNAPSACK

# Hamiltonian Cycle

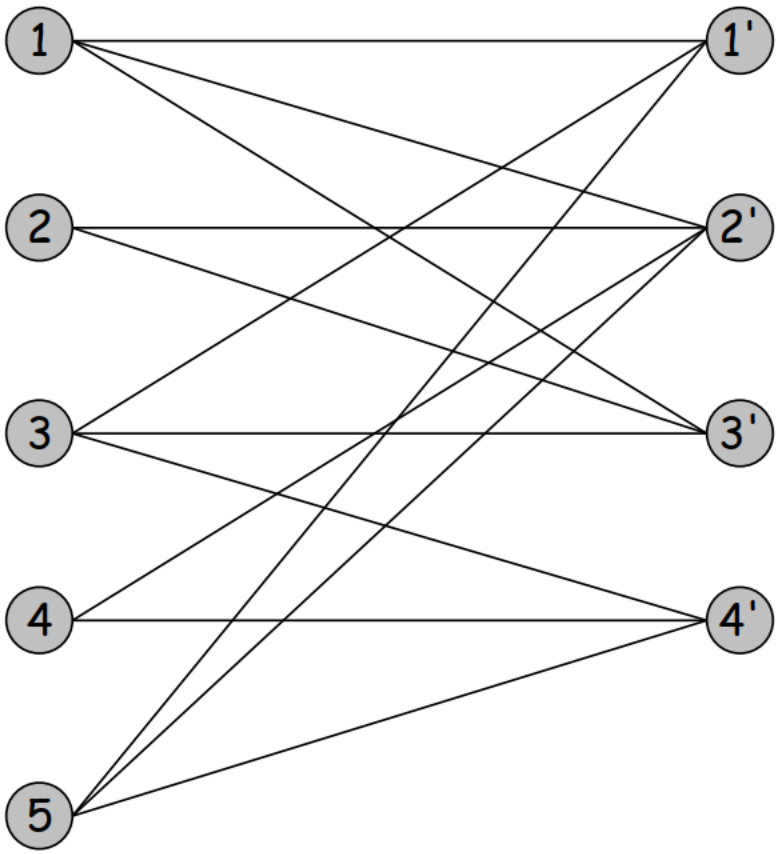
- **HAM-CYCLE:** given an undirected graph G = (V, E), does there exist a simple cycle that contains every node in V



YES: vertices and faces of a dodecahedron

# Hamiltonian Cycle

- **HAM-CYCLE:** given an undirected graph G = (V, E), does there exist a simple cycle that contains every node in V

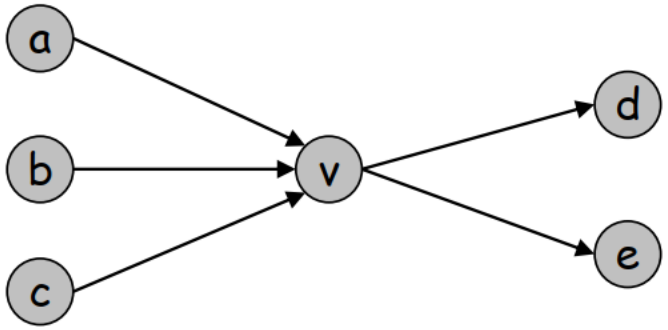



NO: bipartite graph with odd number of nodes

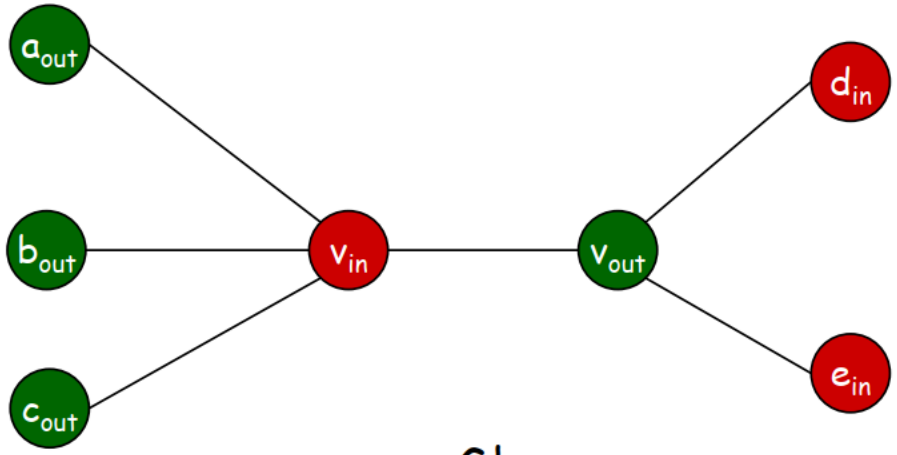
# Directed Hamiltonian Cycle

- **DIR-HAM-CYCLE:** given a **digraph** G = (V, E), does there exists a simple directed cycle that contains every node in V?

- **Claim.** DIR-HAM-CYCLE $\leq_p$ HAM-CYCLE
- **Pf.** Given a directed graph G = (V, E), construct an undirected graph G'




G



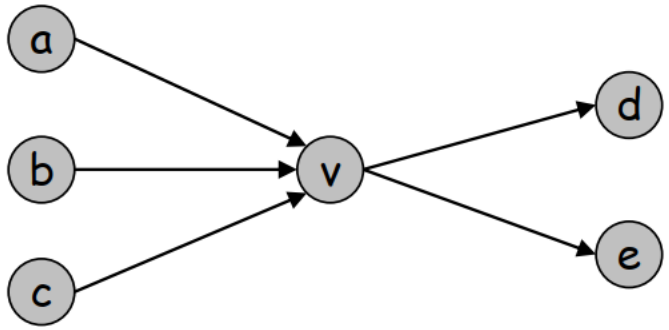

G'

# Directed Hamiltonian Cycle

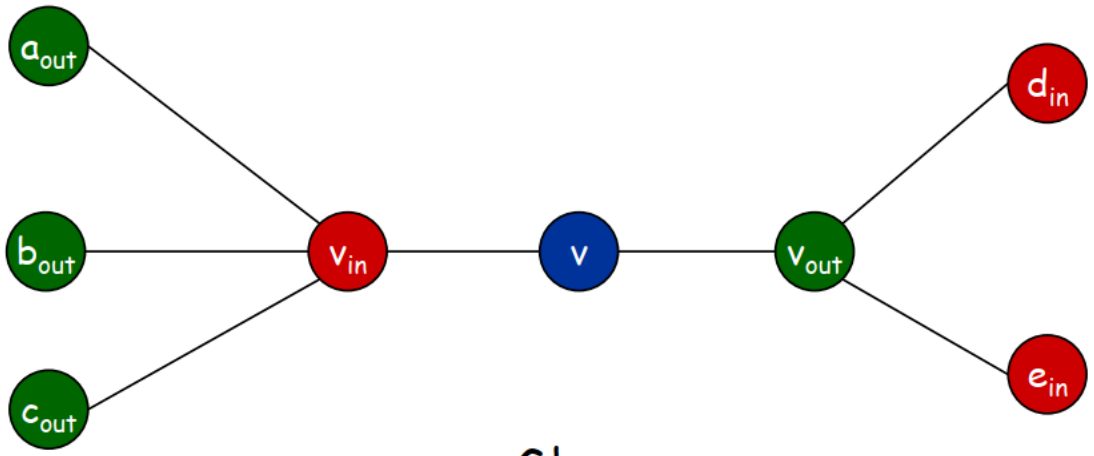
- **DIR-HAM-CYCLE:** given a **digraph** G = (V, E), does there exists a simple directed cycle that contains every node in V?

- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that X $\leq_p$ Y

- **Claim.** DIR-HAM-CYCLE $\leq_p$ HAM-CYCLE
- **Pf.** Given a directed graph G = (V, E), construct an undirected graph G' with 3n nodes




G




G'

# Directed Hamiltonian Cycle

- **Claim.** G has a Hamiltonian cycle iff G' does

- **Pf. →**
  - Suppose G has a directed Hamiltonian cycle
  - Then G' has an undirected Hamiltonian cycle (same order)

- **Pf. ←**
  - Suppose G' has an undirected Hamiltonian cycle
  - The cycle must visit nodes in G' using one of following two orders:
  - …, B, G, R, B, G, R, B, G, R, B, …
  - …, B, R, G, B, R, G, B, R, G, B, …
  - Blue nodes in this cycle make up directed Hamiltonian cycle in G, or reverse of one

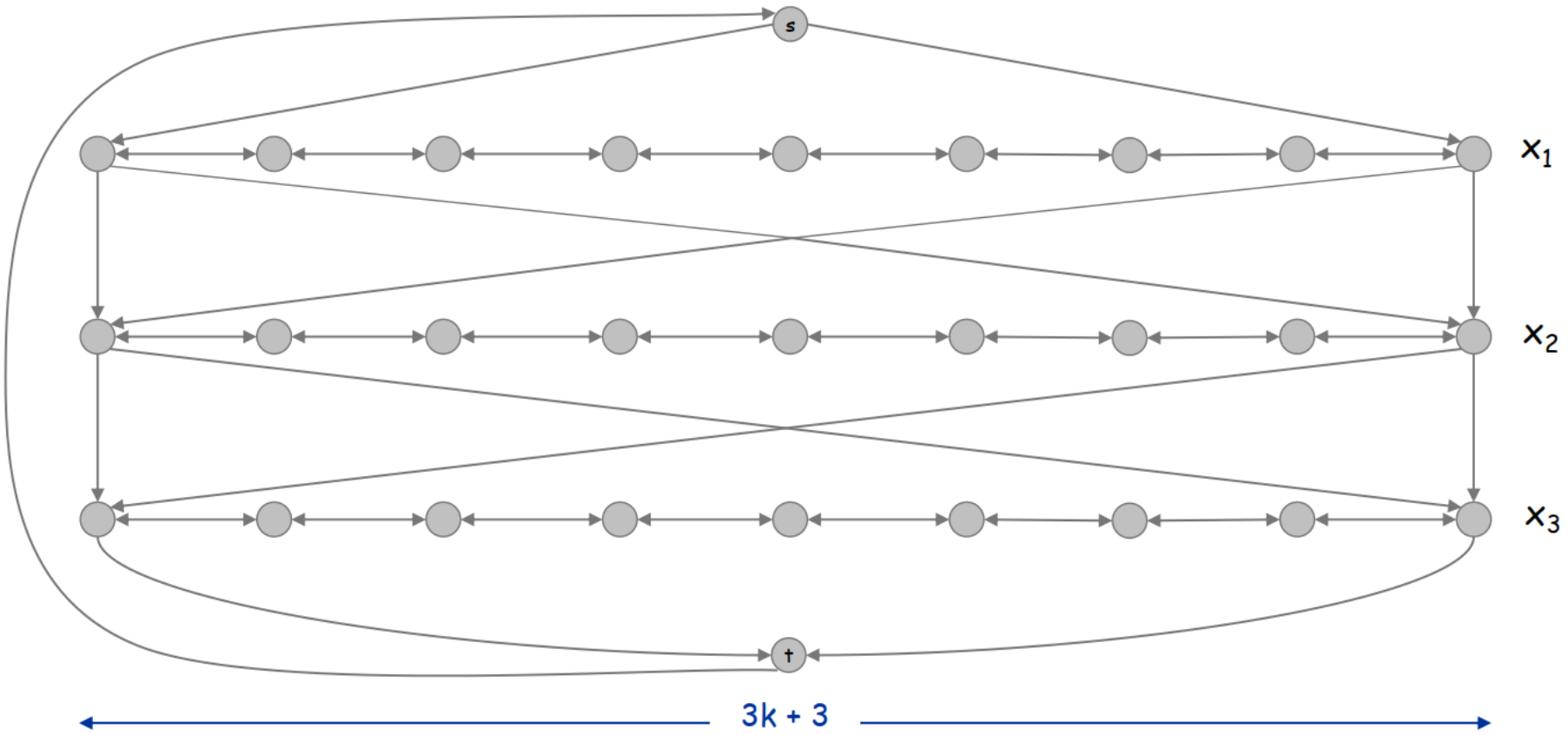
# 3-SAT Reduces to Directed Hamiltonian Cycle

- **Claim.** 3-SAT $\leq_p$ DIR-HAM-CYCLE

- **Pf.** Given an instance $\phi$ of 3-SAT, we construct an instance of DIR-HAM-CYCLE that has a Hamiltonian cycle iff $\phi$ is satisfiable

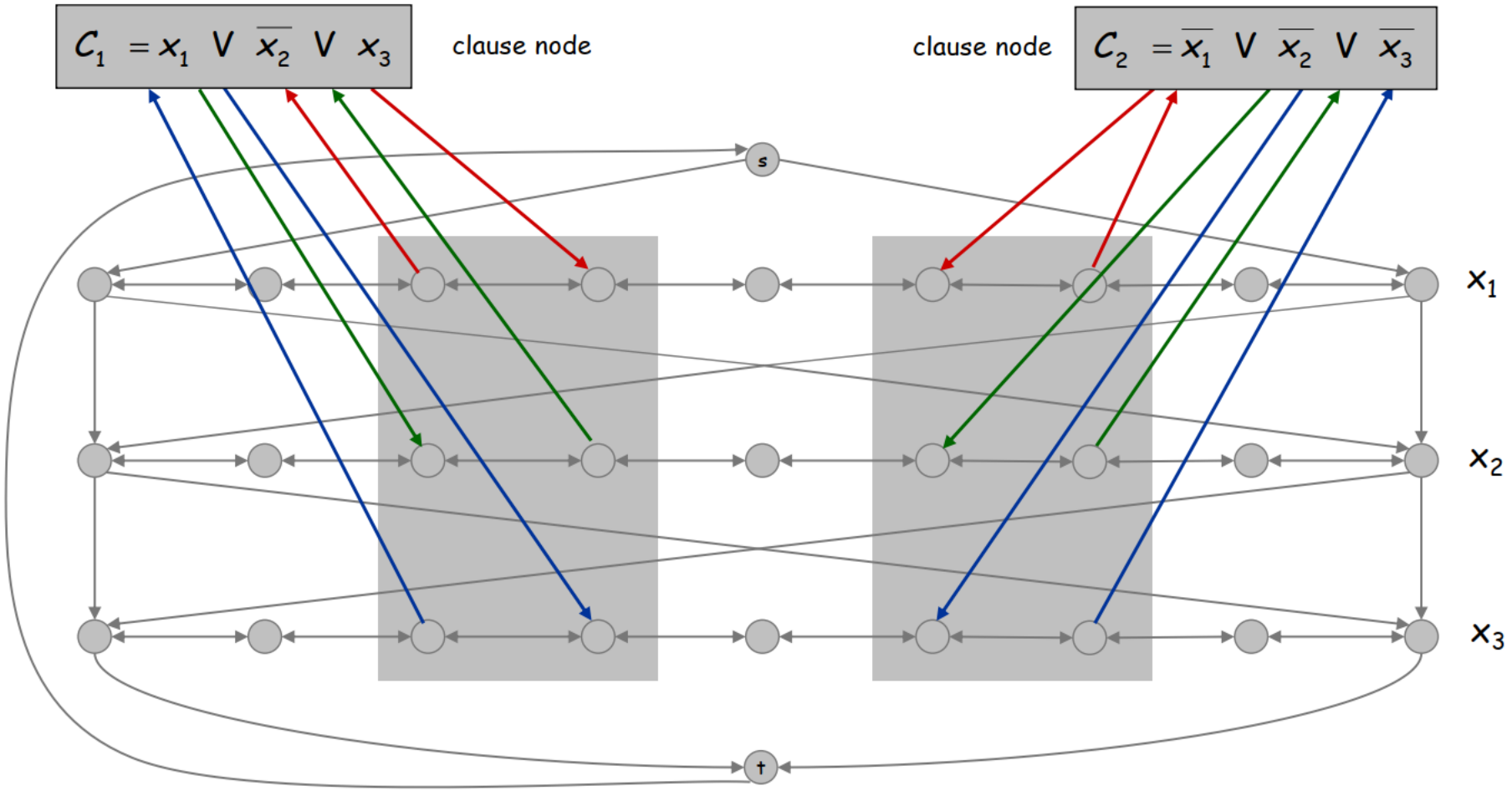
# 3-SAT Reduces to Directed Hamiltonian Cycle

- **Construction. Given 3-SAT instance $\phi$** with n variable $x_i$ and k clauses.
  - Construct G to have $2^n$ Hamiltonian cycles
  - Intuition: traverse path i from left to right $\leftrightarrow$ set variable $x_i = 1$



# 3-SAT Reduces to Directed Hamiltonian Cycle

- **Construction. Given 3-SAT instance $\phi$** with n variable $x_i$ and k clauses.
  - For each clause: add a node and 6 edges



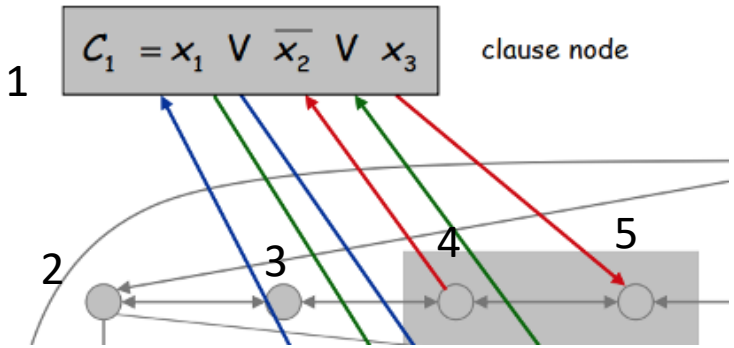
# 3-SAT Reduces to Directed Hamiltonian Cycle



- **Claim.** $\phi$ is satisfiable iff G has a Hamiltonian cycle
- **Pf. →**
  - Suppose 3-SAT instance has satisfying assignment $x^*$
  - Then, define Hamiltonian cycle in G as follows:
    - If $x^*_i = 1$, traverse row i from left to right
    - If $x^*_i = 0$, traverse row i from right to left
    - For each clause $C_j$, there will be at least one row i in which we are going in "correct" direction to splice node $C_j$ into tour

# 3-SAT Reduces to Directed Hamiltonian Cycle

- **Claim.** $\phi$ is satisfiable iff G has a Hamiltonian cycle
- **Pf. ←**
  - Suppose G has a Hamiltonian cycle T
  - If the cycle T enters clause node $C_j$, it must depart on mate edge
    - Thus, nodes immediately before and after $C_j$ are connected by an edge e in G
    - Removing $C_j$ from cycle, and replacing it with edge e yields Hamiltonian cycle on G – { $C_j$ }
    - Continuing in this way, we are left with Hamiltonian cycle T' in G – { $C_1$, $C_2$, …, $C_k$}
    - Set $x^*_i = 1$ iff T' traverse row i left to right
    - Since T' visits each clause node $C_j$, at least one of the paths is traversed in "correct" direction, and each clause is satisfied



# Next Time:
# NP and Computational Intractability (Cont.)